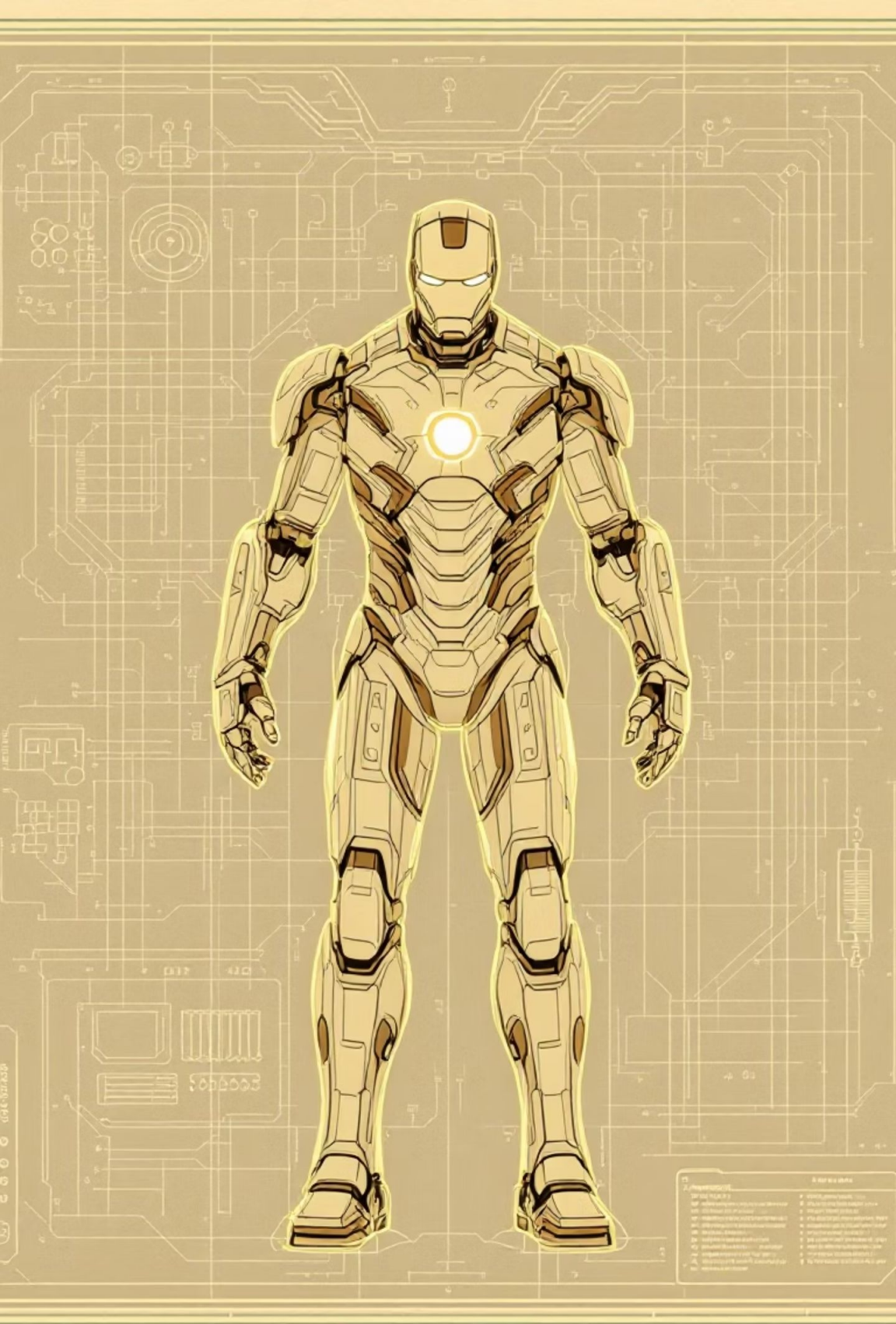




# Abstract Classes & Interfaces

# Your Mission: Design Iron Man Suits

Imagine you've just been hired by Stark Industries! Your task: design a fleet of Iron Man suits using Java. But Tony, ever the innovator, presents a challenge:



1

## Common Rules

All suits must adhere to a core set of functionalities.



2

## Unique Operations

Each suit needs its own distinct operational characteristics.

How do we tackle this engineering marvel in Java? With Abstract Classes and Interfaces! Let's get started.



# The Blueprint: Abstract Class for Iron Man Suits

## Why an Abstract Class?

Tony declares: "Every Iron Man suit must be able to fly and attack. But the Mark-1, Mark-42, and Nanosuit will all behave differently."

This is where an abstract class becomes our foundational blueprint. It establishes the mandatory actions without dictating their exact execution.

# Abstract Class: Defining the Core Functions

```
abstract class IronManSuit {  
    abstract void fly();  
    abstract void attack();  
    void powerOn() {  
        System.out.println("Suit powered on");  
    }  
}
```

This IronManSuit abstract class serves as the base. It defines that all suits must have fly() and attack() methods, but their implementation is left to specific suit models. The powerOn() method, however, is a common functionality already implemented.

1

## Defines Rules

Establishes essential behaviors for all subclasses.

2

## Incomplete Implementation

Does not provide full method bodies for abstract methods.

3

## No Direct Objects

Cannot be instantiated directly; it's a concept, not a concrete item.





## Bringing Suits to Life: Implementing Abstract Methods

```
class Mark42 extends IronManSuit {  
    void fly() {  
        System.out.println("Mark 42 flying using jet boosters");  
    }  
    void attack() {  
        System.out.println("Mark 42 firing repulsor beams");  
    }  
}
```

Here, the Mark42 class extends IronManSuit and provides concrete implementations for fly() and attack(). Without these implementations, Java would produce a compilation error, ensuring every concrete suit model fulfills its basic requirements.

```
public class Main {  
    public static void main(String[] args) {  
        IronManSuit suit = new Mark42();  
        suit.powerOn();  
        suit.fly();  
        suit.attack();  
    }  
}
```

While you cannot create an object directly from an abstract class, you can create a reference to it, pointing to a concrete subclass object.

# Expanding Capabilities: Interfaces for Suit Abilities

## Why Interfaces?

Tony now says: "Some suits can swim, some can hack, and others need to go to space." These are specialized abilities that aren't inherent to every single suit.

For these distinct abilities, we use interfaces.

```
interface Flyable {  
    void fly();  
}
```

An interface defines a contract: it specifies what to do, but not how to do it. It's a pure declaration of capability.

```
class NanoSuit implements Flyable {  
    public void fly() {  
        System.out.println("Nano suit flying using nano thrusters");  
    }  
}
```

When a class implements an interface, it must provide a public implementation for all its methods.

# Interface Power: Constants and Multiple Inheritance

## Interface Variables (Constants)

```
interface SuitPower {  
    int MAX_POWER = 100;  
}
```

Any variables declared in an interface are implicitly public, static, and final (constants). They represent universal, unchangeable values.

## Multiple Interfaces (Exam Gold Point!)

Java does not support multiple inheritance for classes, but what if Tony needs suits with multiple, distinct abilities? This is where interfaces shine!

```
interface Flyable { void fly(); }  
interface Weaponized { void fireWeapon(); }  
  
class Mark85 implements Flyable, Weaponized {  
    public void fly() { System.out.println("Mark 85 flying"); }  
    public void fireWeapon() { System.out.println("Mark 85 firing nano weapons"); }  
}
```

A class can implement multiple interfaces, allowing it to inherit diverse sets of behaviors. This is a critical concept for your exams!

# Abstract Class vs. Interface: The Stark Industries Comparison

| Example              | Base Suit Blueprint | Suit Power Module   |
|----------------------|---------------------|---------------------|
| Methods              | Abstract + Concrete | Abstract Only       |
| Variables            | Any type            | public static final |
| Constructor          | Yes                 | No                  |
| Multiple Inheritance | ✗ No                | ✓ Yes               |
| Keyword              | extends             | implements          |

Think of it this way: an abstract class is the fundamental design of an Iron Man suit, providing both essential structure and flexible parts. An interface is a swappable power module, granting specific abilities like flight or weaponry that can be added to any compatible suit.





# Practical Application: The Hulkbuster Attack

Let's look at an example often seen in exams. This simple program demonstrates the core concept of implementing an abstract method.

```
abstract class IronManSuit {  
    abstract void attack();  
}  
class Hulkbuster extends IronManSuit {  
    void attack() {  
        System.out.println("Hulkbuster heavy punch  
attack");  
    }  
}
```

The Hulkbuster class, extending IronManSuit, is compelled to provide its unique attack() implementation – in this case, a powerful punch. This type of focused program is perfect for board exams.